MINI PROJECT REPORT

Used Car Price Prediction System Using Machine Learning

1. Abstract

Estimating the resale value of a pre-owned car is difficult because price depends on non-linear interactions between age, usage, engine specification, brand and accident history. This project builds an end-to-end machine learning application that predicts the market value of a used car from nine vehicle attributes. A dataset of 4,009 used car listings was cleaned, price outliers were removed with the IQR rule, and engine specifications were extracted from text. Numeric features were standardised, categorical features were one-hot encoded, and the target price was log-transformed to reduce skew. Linear Regression, Ridge Regression, Lasso Regression and Gradient Boosting Regressor were compared using R^2, RMSE and MAE. Gradient Boosting performed best, with a test R^2 of 0.82, an RMSE of about $8,821 and an MAE of about $5,994, against an R^2 of about 0.69 for the linear models. It was packaged with its preprocessing into a single Scikit-Learn pipeline and deployed as a Streamlit web application that returns an estimate in both USD and INR.

2. Introduction

The pre-owned vehicle market has grown quickly with the spread of online marketplaces. Unlike a new car with a fixed list price, the price of a used car depends on depreciation, physical wear, engine and performance specifications, brand positioning and past accidents. Setting a fair price by judgement alone often leads to overpricing, underpricing or long negotiation.

Machine learning can learn these pricing patterns from historical listings. This project develops a complete workflow: data cleaning and exploratory analysis, feature engineering, training and benchmarking of regression models, selection of the best model, and deployment as an interactive web application that anyone can use without technical knowledge.

3. Problem Statement

Buyers and sellers of used cars lack a quick, objective way to estimate a fair price. Manual valuation is subjective, and simple rules cannot capture how factors such as mileage, engine power and brand tier interact. The objective of this project is to build a regression model that predicts the resale price of a used car from its age, mileage, horsepower, engine size, cylinders, fuel type, transmission, brand tier and accident history, with the smallest possible error against actual prices, and to make it available through a simple web interface.

4. Literature Survey

Recent studies on used car price prediction consistently report that ensemble and boosting models outperform linear models on tabular vehicle data. Five relevant papers are summarised below.

No.	Authors and year	Methods	Key finding
1	Bhatnagar, Lokesh, Shreyas, Flammini and Gautam (2024)	Voting, Gradient Boosting, Random Forest, Decision Tree and Support Vector regressors with GridSearchCV tuning	Compared ensemble regressors for car price prediction and found the voting regressor performed best.
2	Y. Wang (2024)	Linear Regression, Decision Tree, Random Forest and XGBoost on 17,966 Ford listings with nine features	XGBoost and Random Forest beat traditional models; XGBoost achieved the highest R^2 of 0.9347.
3	A. T. Amshi (2023)	KNN, Linear Regression, Decision Tree and Gradient Boosting on a Kaggle used-vehicle dataset	Gradient Boosting had the lowest error and highest R^2 among the compared models.
4	A. Jamalamadaka (2025)	11 machine learning models and 3 neural networks on an 8,128-record CarDekho dataset, with imputation and feature engineering	LightGBM and CatBoost were the best models by R^2; tree-based models beat deep learning on this tabular data.
5	Shakti Dheerays, Gopakumar, Keerthana and Kokatnoor	Logistic Regression, Random Forest, Linear Regression, SVM and Gradient Boosting	Random Forest and Gradient Boosting scored the best accuracy (87%), at a higher training time than linear models.

Table 1: Summary of related work

These studies support the choice made in this project: a linear baseline for reference, regularized linear models for comparison, and a Gradient Boosting model as the main non-linear learner.

5. Technical Details

5.1 Software (Front end and Back end)

- **Front end:** Streamlit web application (app.py) with dropdowns, sliders and number inputs, and a sidebar for the USD to INR exchange rate.
- **Back end:** Python 3.10+ with a Scikit-Learn pipeline saved as model.joblib.
- **Tools:** Jupyter Notebook for analysis and training, Git and GitHub for version control, Streamlit Community Cloud for deployment.

5.2 Dataset Description

Link: https://www.kaggle.com/datasets/taeefnajib/used-car-price-prediction-dataset

File and size: used_cars.csv, containing 4,009 used car listings with 12 columns (US market, prices in USD). The raw columns are brand, model, model_year, milage, fuel_type, engine, transmission, ext_col, int_col, accident, clean_title and price. Fuel type had 170 missing values, accident history 113 and clean title 596. After removing 244 price outliers, 3,765 records remained for modelling.

The model uses the nine features below, derived from these raw columns, and predicts the price.

Feature	Type	Description
age	Numeric	Vehicle age, calculated as 2026 minus the model year
milage	Numeric	Total miles driven (column name as spelled in the dataset)
hp	Numeric	Engine horsepower, extracted from the engine description
engine_l	Numeric	Engine displacement in litres, extracted from the engine description
cylinders	Numeric	Number of cylinders, extracted from the engine description
fuel_type	Categorical	Gasoline, Diesel, Hybrid, Plug-In Hybrid, E85 Flex Fuel or Unknown
transmission	Categorical	Automatic or Manual (simplified from 62 raw labels)
brand_tier	Categorical	Economy, Luxury or Exotic
accident	Binary	Accident or damage history: 0 = none reported, 1 = reported
price	Target	Resale price in USD (modelled as log(1 + price))

Table 2: Features used by the model

5.3 Data Preprocessing Techniques

- **Text cleaning:** currency symbols and commas were removed from price, and the text "mi." and commas were removed from mileage, converting both to numbers.
- **Missing values:** missing fuel types were labelled "Unknown". Missing hp, engine size and cylinder values were filled with the column median.
- **Outlier removal:** rows with prices outside 1.5 times the interquartile range were removed (244 rows).
- **Feature extraction:** regular expressions pulled horsepower, engine size in litres and cylinder count out of the engine text.
- **Feature engineering:** age was calculated as 2026 minus the model year. Brands were grouped into Economy, Luxury and Exotic tiers. Transmission labels were simplified to Automatic or Manual, and accident history was converted to a 0/1 flag.
- **Scaling and encoding:** StandardScaler on age, mileage, hp, engine size and cylinders; OneHotEncoder on fuel type, transmission, brand tier and accident status.
- **Target transformation:** price was modelled as log(1 + price) to reduce right skew, and predictions are converted back with expm1 (Figure 1).
- **Data splitting:** 80% training and 20% testing (random_state = 42), done before fitting any transformer to avoid data leakage.
- **Dimensionality check:** PCA on the scaled numeric features showed that 4 components explain 95% of the variance, so all numeric features were retained.

5.4 Libraries

pandas 3.0.6 and NumPy 2.5.4 for data handling; matplotlib and seaborn for visualisation; re for text parsing; scikit-learn 1.9.1 for preprocessing, modelling and evaluation; joblib 1.6.0 for saving the model; Streamlit 1.66.0 for the web interface.

5.5 Functions Used

- Data handling: read_csv(), info(), describe(), str.replace(), str.extract(), fillna(), quantile(), apply(), np.log1p(), np.expm1()
- Preprocessing: ColumnTransformer(), StandardScaler(), OneHotEncoder(), PCA(), Pipeline(), train_test_split()
- Modelling: LinearRegression(), Ridge(), Lasso(), GradientBoostingRegressor(), GridSearchCV(), fit(), predict()
- Evaluation: mean_absolute_error(), mean_squared_error(), r2_score(), feature_importances_
- Deployment: joblib.dump(), joblib.load(), and Streamlit functions such as selectbox(), number_input(), radio(), button() and metric()

6. Algorithm

The project uses **supervised regression**, where the model learns a mapping from vehicle features to a continuous price. Four algorithms were benchmarked:

- **Linear Regression:** baseline parametric model that assumes a linear relationship between features and price.
- **Ridge Regression:** linear model with an L2 penalty that shrinks coefficients and reduces the effect of correlated features. Alpha is tuned with 5-fold GridSearchCV over 20 values from 0.001 to 1000, scored on R^2.
- **Lasso Regression:** linear model with an L1 penalty that can drive weak coefficients to zero, acting as feature selection. Alpha is tuned in the same way as Ridge.
- **Gradient Boosting Regressor (selected):** an ensemble of shallow decision trees built one after another, where each new tree is fitted to the errors of the previous ones. It captures non-linear effects and interactions between features without manual feature crossing. The final model uses 100 trees, a learning rate of 0.1 and a maximum depth of 4.

Preprocessing and the chosen model are bundled in one Scikit-Learn Pipeline, so the same transformations are applied during training and during prediction in the web app.

7. Performance Metrics

Metric	Formula	Meaning
MAE	$(1/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$	Average absolute error in price units
MSE	$(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$	Average squared error; penalises large errors
RMSE	$\sqrt{\text{MSE}}$	Error in the same unit as price

Metric	Formula	Meaning
R^2 score	$1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$	Share of price variance explained by the model

Table 3: Evaluation metrics

8. Comparison of Algorithms

All four models were trained on the same 80% training split and evaluated on the same 20% test set. Predictions were converted back to dollars with expm1 before computing the metrics, so the errors are in USD.

Algorithm	Test R^2	RMSE ($)	MAE ($)
Linear Regression	0.6912	11,633.89	7,553.56
Ridge Regression (tuned)	0.6913	11,633.15	7,552.80
Lasso Regression (tuned)	0.6889	11,677.35	7,552.37
Gradient Boosting (selected)	**0.8225**	**8,820.52**	**5,994.24**

Table 4: Model comparison on the test set

The three linear models perform almost identically (R^2 about 0.69), so regularization added little. This suggests the limit comes from the linear form, not from overfitting. Gradient Boosting raises R^2 to 0.82, and reduces RMSE by about 24% and MAE by about 21% compared with Linear Regression. This is consistent with the literature, where boosted tree models outperform linear models on tabular vehicle data, because price changes non-linearly with age, mileage and brand tier.

9. Results and Visualizations

Figure 1 shows that the original price distribution is strongly right-skewed, while the log-transformed price is much closer to normal. This supports modelling log(1 + price).

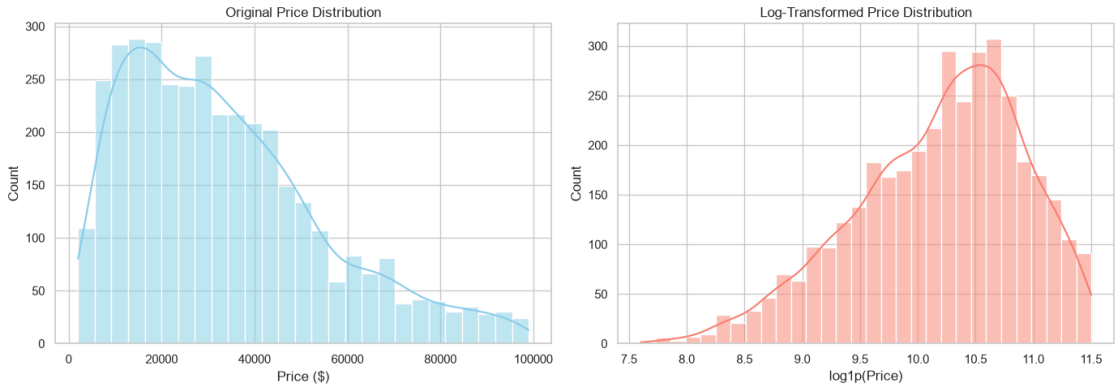
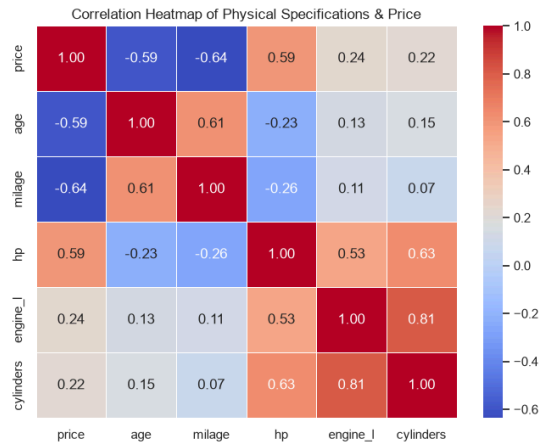


Figure 1: Price distribution before and after log transformation

The correlation heatmap (Figure 2) shows that price is strongly negatively correlated with mileage (−0.64) and age (−0.59), and positively correlated with horsepower (0.59). Engine size and cylinders are strongly correlated with each other (0.81). The residual plot (Figure 3) shows errors scattered around zero for the Gradient Boosting model, with larger errors at higher prices.



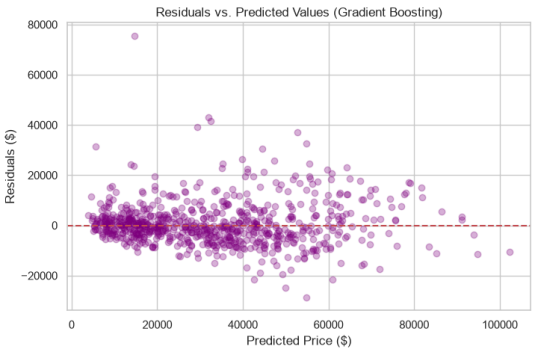


Figure 2 (left): Correlation heatmap. *Figure 3 (right): Residuals vs. predicted price (Gradient Boosting)*

The feature importance chart (Figure 4) shows that mileage (about 36%), age (about 33%) and horsepower (about 21%) account for most of the model's predictive power, followed by engine size (about 6%). Brand tier, fuel type and cylinders contribute only small amounts.

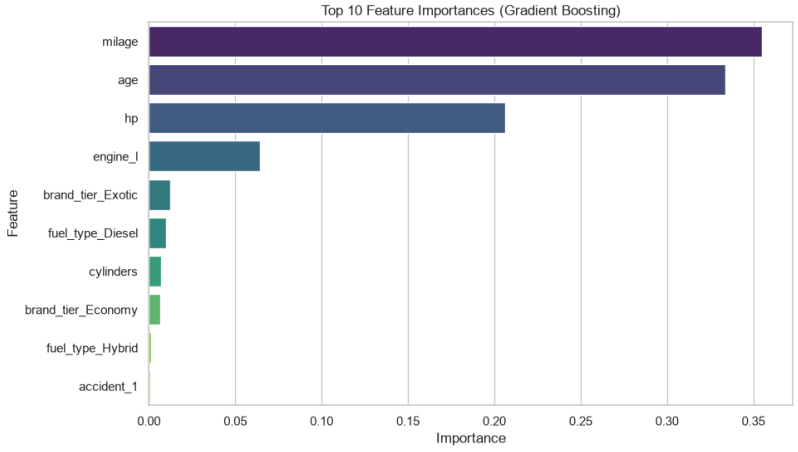


Figure 4: Top 10 feature importances (Gradient Boosting)

Web application

The Streamlit application lets the user choose the brand, fuel type and transmission, enter model year, mileage, horsepower, engine size and cylinders, and select the accident history. It returns an instant estimate in USD and INR with an adjustable exchange rate (default 85). An expandable panel shows the exact feature vector passed to the model. Because the training data is US-based, the estimate is a US-market guide and is less reliable for rare or exotic models.

Future scope: price confidence intervals using quantile regression, SHAP-based explanations, automatic spec pre-fill for common models, a deal rating (great, fair or overpriced) against seller prices, and benchmarking of CatBoost and LightGBM.

10. References

[1] A. T. Amshi, "Vehicle Price Prediction by Aggregating Decision Tree Model with Boosting Model," arXiv:2307.15982, 2023.

[2] P. Bhatnagar, G. H. Lokesh, J. Shreyas, F. Flammini and S. Gautam, "An Analysis of Car Price Prediction using Machine Learning," in Proc. 9th Int. Conf. on Machine Learning Technologies (ICMLT), 2024, doi: 10.1145/3674029.3674032.

[3] Y. Wang, "Price Prediction of Ford Cars Applying Multiple Machine Learning Methods," SciTePress, 2024.

[4] A. Jamalamadaka, "Practical Recommendations for Machine Learning-Based Tabular Regression Concerning Car Price Prediction," The National High School Journal of Science, 2025.

[5] S. Shakti Dheerays, A. Gopakumar, B. Keerthana and S. A. Kokatnoor, "AI-Powered Wheels: Machine Learning Approaches for Predicting Used Car Prices," conference paper, CHRIST (Deemed to be University) Institutional Repository.

[6] AnulaRB, "Used Car Price Predictor," GitHub repository, github.com/AnulaRB/used-car-price-predictor.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.